

Lab 9: SVM and PCA

Aim: Implement SVM for classification and PCA for dimensionality reduction using real-world datasets.

Objectives

- Train and tune a linear-kernel SVM.
- Evaluate it using standard classification metrics.
- Reduce high-dimensional data using PCA and study retained variance.
- Compare PCA with supervised LDA.

```
In [1]: import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer, load_wine
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.decomposition import PCA
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, silhouette_score
)
from sklearn.model_selection import GridSearchCV, cross_val_score, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

RANDOM_STATE = 42
sns.set_theme(style="whitegrid", context="notebook")
pd.set_option("display.max_columns", 15)
```

Part A: Support Vector Machine (SVM)

Dataset: [UCI Breast Cancer Wisconsin \(Diagnostic\)](#). The scikit-learn copy is used so the notebook runs without a network connection.

```
In [2]: cancer = load_breast_cancer(as_frame=True)
X_cancer = cancer.data
y_cancer = cancer.target

cancer_summary = pd.DataFrame({
    "Value": [X_cancer.shape[0], X_cancer.shape[1], int(X_cancer.isna().sum()),
    ], index=["Samples", "Features", "Missing values", "Classes (encoded 0, 1)"]
})
```

```
display(cancer_summary)
display(X_cancer.head())
print("Class counts:")
display(y_cancer.map(dict(enumerate(cancer.target_names))).value_counts().to
```

	Value
Samples	569
Features	30
Missing values	0
Classes (encoded 0, 1)	[malignant, benign]

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	...	worst area
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	...	2019.0
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	...	1956.0
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	...	1709.0
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	...	567.7
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	...	1575.0

5 rows × 30 columns

◀  ▶

Class counts:

	count
target	
benign	357
malignant	212

Preprocessing and 80:20 split

- The dataset has no missing values.
- Stratification keeps the class ratio similar in both sets.
- Standardization is fitted only on training folds through a pipeline.

```
In [3]: X_train, X_test, y_train, y_test = train_test_split(
        X_cancer, y_cancer, test_size=0.20, stratify=y_cancer, random_state=RAND
    )
print(f"Training samples: {len(X_train)} | Testing samples: {len(X_test)}")

# Brief kernel comparison on training data only. The required final model re
kernel_results = []
```

```

for kernel in ["linear", "rbf", "poly", "sigmoid"]:
    candidate = Pipeline([
        ("scaler", StandardScaler()),
        ("svm", SVC(kernel=kernel, C=1)),
    ])
    scores = cross_val_score(candidate, X_train, y_train, cv=5, scoring="f1_
    kernel_results.append({"Kernel": kernel, "Mean CV Macro F1": scores.mean
kernel_comparison = pd.DataFrame(kernel_results).sort_values("Mean CV Macro
display(kernel_comparison.style.format({"Mean CV Macro F1": "{:.4f}", "CV St

svm_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel="linear"))
])

param_grid = {
    "svm__C": [0.01, 0.1, 1, 10, 100],
    "svm__class_weight": [None, "balanced"],
}
grid_search = GridSearchCV(
    svm_pipeline, param_grid, cv=5, scoring="f1_macro", n_jobs=-1, return_tr
)
grid_search.fit(X_train, y_train)

cv_results = pd.DataFrame(grid_search.cv_results_)
    ["param_svm__C", "param_svm__class_weight", "mean_train_score", "mean_te
].sort_values("rank_test_score")
display(cv_results.head(10).style.format({"mean_train_score": "{:.4f}", "mea
print("Best parameters:", grid_search.best_params_)
print(f"Best mean cross-validation macro F1: {grid_search.best_score_:.4f}")

```

Training samples: 455 | Testing samples: 114

	Kernel	Mean CV Macro F1	CV Std. Dev.
1	rbf	0.9694	0.0193
0	linear	0.9646	0.0150
3	sigmoid	0.9641	0.0110
2	poly	0.8813	0.0223

	param_svm_C	param_svm_class_weight	mean_train_score	mean_test_score	rank_test_score
3	0.100000	balanced	0.9853	0.9764	
2	0.100000	nan	0.9841	0.9762	
1	0.010000	balanced	0.9817	0.9741	
0	0.010000	nan	0.9756	0.9712	
5	1.000000	balanced	0.9883	0.9670	
4	1.000000	nan	0.9894	0.9646	
6	10.000000	nan	0.9941	0.9625	
7	10.000000	balanced	0.9936	0.9625	
8	100.000000	nan	0.9988	0.9417	
9	100.000000	balanced	0.9982	0.9417	

Best parameters: {'svm_C': 0.1, 'svm_class_weight': 'balanced'}
Best mean cross-validation macro F1: 0.9764

```
In [4]: best_svm = grid_search.best_estimator_
y_pred = best_svm.predict(X_test)

# Malignant (label 0) is treated as the positive class because detecting it
svm_metrics = pd.DataFrame({
    "Score": [
        accuracy_score(y_test, y_pred),
        precision_score(y_test, y_pred, pos_label=0),
        recall_score(y_test, y_pred, pos_label=0),
        f1_score(y_test, y_pred, pos_label=0),
    ]
}, index=["Accuracy", "Precision (malignant)", "Recall (malignant)", "F1 Score"])
display(svm_metrics.style.format("{:.4f}"))

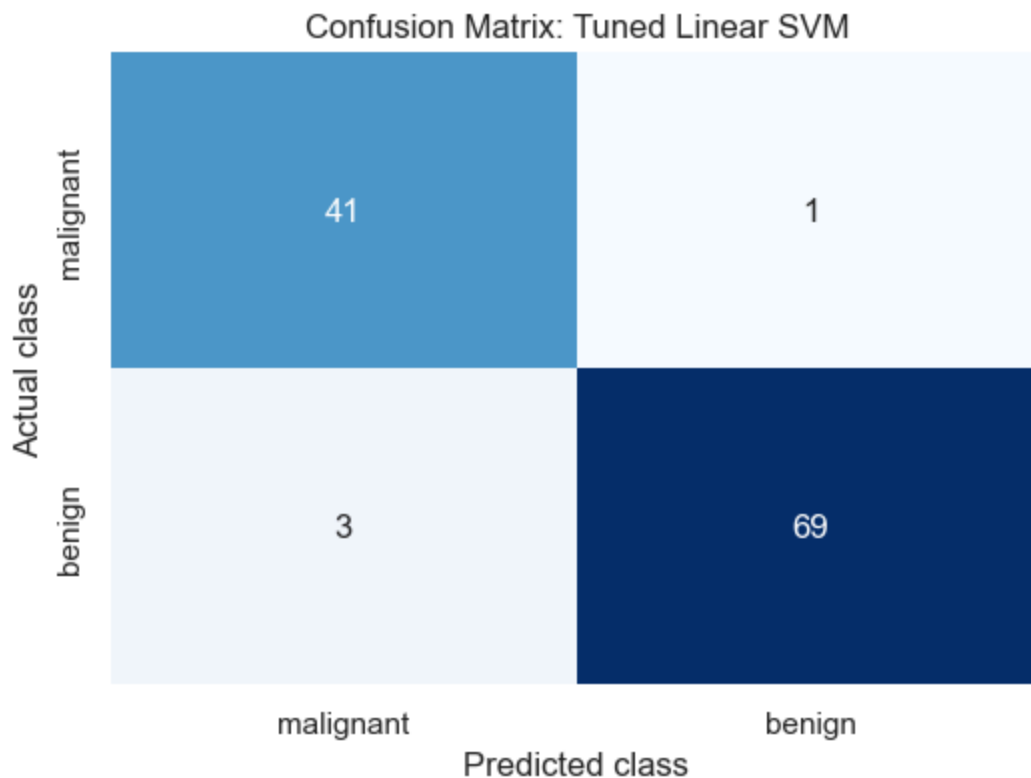
print("Classification report:")
print(classification_report(y_test, y_pred, target_names=cancer.target_names))

cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
plt.figure(figsize=(5.5, 4.2))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=cancer.target_names, yticklabels=cancer.target_names)
plt.title("Confusion Matrix: Tuned Linear SVM")
plt.xlabel("Predicted class")
plt.ylabel("Actual class")
plt.tight_layout()
plt.show()
```

	Score
Accuracy	0.9649
Precision (malignant)	0.9318
Recall (malignant)	0.9762
F1 Score (malignant)	0.9535

Classification report:

	precision	recall	f1-score	support
malignant	0.9318	0.9762	0.9535	42
benign	0.9857	0.9583	0.9718	72
accuracy			0.9649	114
macro avg	0.9588	0.9673	0.9627	114
weighted avg	0.9659	0.9649	0.9651	114



```
In [5]: tn_m, fp_m, fn_m, tp_m = cm.ravel() # Matrix order is [malignant, benign].
print("SVM observations:")
print(f"- Standardization helps SVM handle features with different scales.")
print(f"- At C=1, {kernel_comparison.iloc[0]['Kernel']} had the highest cross")
print(f"- Tuning selected C={grid_search.best_params_['svm__C']} and class_w")
print(f"- Test accuracy is {svm_metrics.loc['Accuracy', 'Score']:.2%}.")
print(f"- The model correctly detected {cm[0, 0]} of {cm[0].sum()} malignant")
print(f"- The linear boundary performs well, so the standardized classes are")
```

SVM observations:

- Standardization helps SVM handle features with different scales.
- At C=1, rbf had the highest cross-validation macro F1 among the tested kernels.
- Tuning selected C=0.1 and class_weight=balanced.
- Test accuracy is 96.49%.
- The model correctly detected 41 of 42 malignant cases.
- The linear boundary performs well, so the standardized classes are close to linearly separable.

Part B: Principal Component Analysis (PCA)

Dataset: [UCI Wine](#). The scikit-learn copy contains the same 13 numeric chemical features and three wine classes.

```
In [6]: wine = load_wine(as_frame=True)
X_wine = wine.data
y_wine = wine.target

wine_summary = pd.DataFrame({
    "Value": [X_wine.shape[0], X_wine.shape[1], int(X_wine.isna().sum().sum()),
], index=["Samples", "Original features", "Missing values", "Classes"])
display(wine_summary)
display(X_wine.head())

wine_scaler = StandardScaler()
X_wine_scaled = wine_scaler.fit_transform(X_wine)
print("Mean after scaling (approximately 0):", np.abs(X_wine_scaled.mean(axis=0)))
print("Standard deviation after scaling (approximately 1):", X_wine_scaled.std(axis=0))
```

Value	
Samples	178
Original features	13
Missing values	0
Classes	3

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	no
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	

Mean after scaling (approximately 0): 0.0

Standard deviation after scaling (approximately 1): 1.0

Explained variance and the 95% threshold

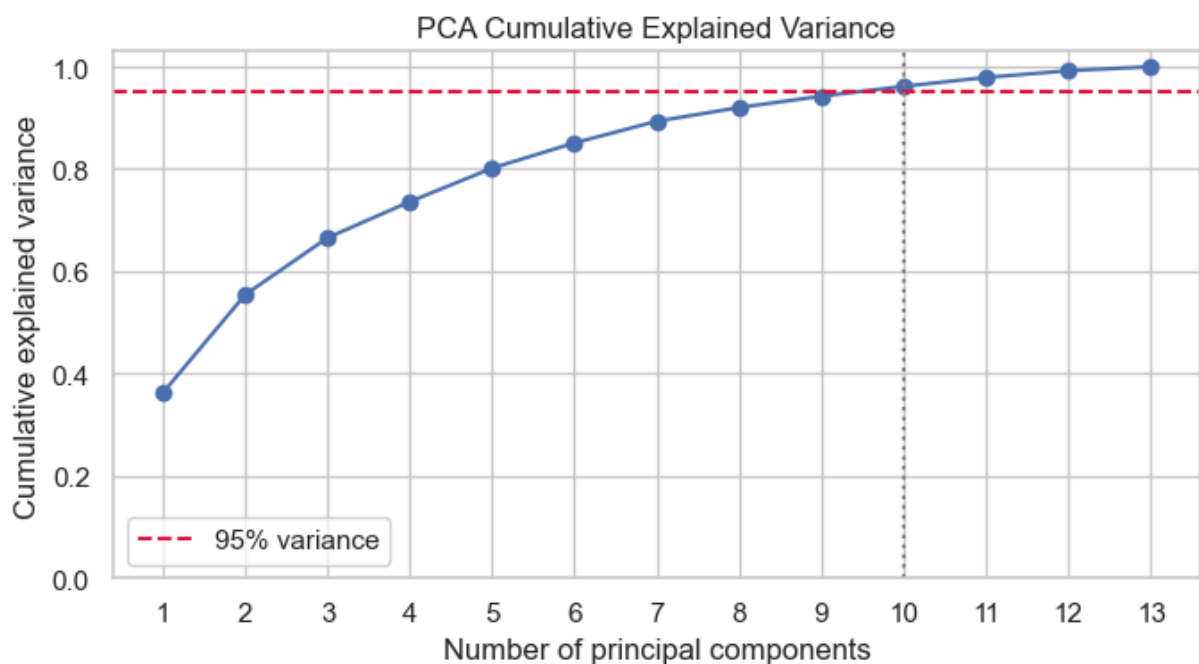
```
In [7]: pca_all = PCA().fit(X_wine_scaled)
explained = pca_all.explained_variance_ratio_
cumulative = np.cumsum(explained)
min_components_95 = int(np.argmax(cumulative >= 0.95) + 1)

variance_table = pd.DataFrame({
    "Principal Component": [f"PC{i}" for i in range(1, len(explained) + 1)],
    "Explained Variance Ratio": explained,
    "Cumulative Variance": cumulative,
})
display(variance_table.style.format({
    "Explained Variance Ratio": "{:.4f}", "Cumulative Variance": "{:.4f}"
}))
print(f"Minimum components needed to retain at least 95% variance: {min_comp

plt.figure(figsize=(7, 4))
plt.plot(range(1, len(cumulative) + 1), cumulative, marker="o")
plt.axhline(0.95, color="crimson", linestyle="--", label="95% variance")
plt.axvline(min_components_95, color="gray", linestyle=":")
plt.xticks(range(1, len(cumulative) + 1))
plt.ylim(0, 1.03)
plt.xlabel("Number of principal components")
plt.ylabel("Cumulative explained variance")
plt.title("PCA Cumulative Explained Variance")
plt.legend()
plt.tight_layout()
plt.show()
```

	Principal Component	Explained Variance Ratio	Cumulative Variance
0	PC1	0.3620	0.3620
1	PC2	0.1921	0.5541
2	PC3	0.1112	0.6653
3	PC4	0.0707	0.7360
4	PC5	0.0656	0.8016
5	PC6	0.0494	0.8510
6	PC7	0.0424	0.8934
7	PC8	0.0268	0.9202
8	PC9	0.0222	0.9424
9	PC10	0.0193	0.9617
10	PC11	0.0174	0.9791
11	PC12	0.0130	0.9920
12	PC13	0.0080	1.0000

Minimum components needed to retain at least 95% variance: 10



```
In [8]: pca_2 = PCA(n_components=2)
X_pca = pca_2.fit_transform(X_wine_scaled)
pca_df = pd.DataFrame(X_pca, columns=["PC1", "PC2"])
pca_df["Class"] = y_wine.map(dict(enumerate(wine.target_names)))

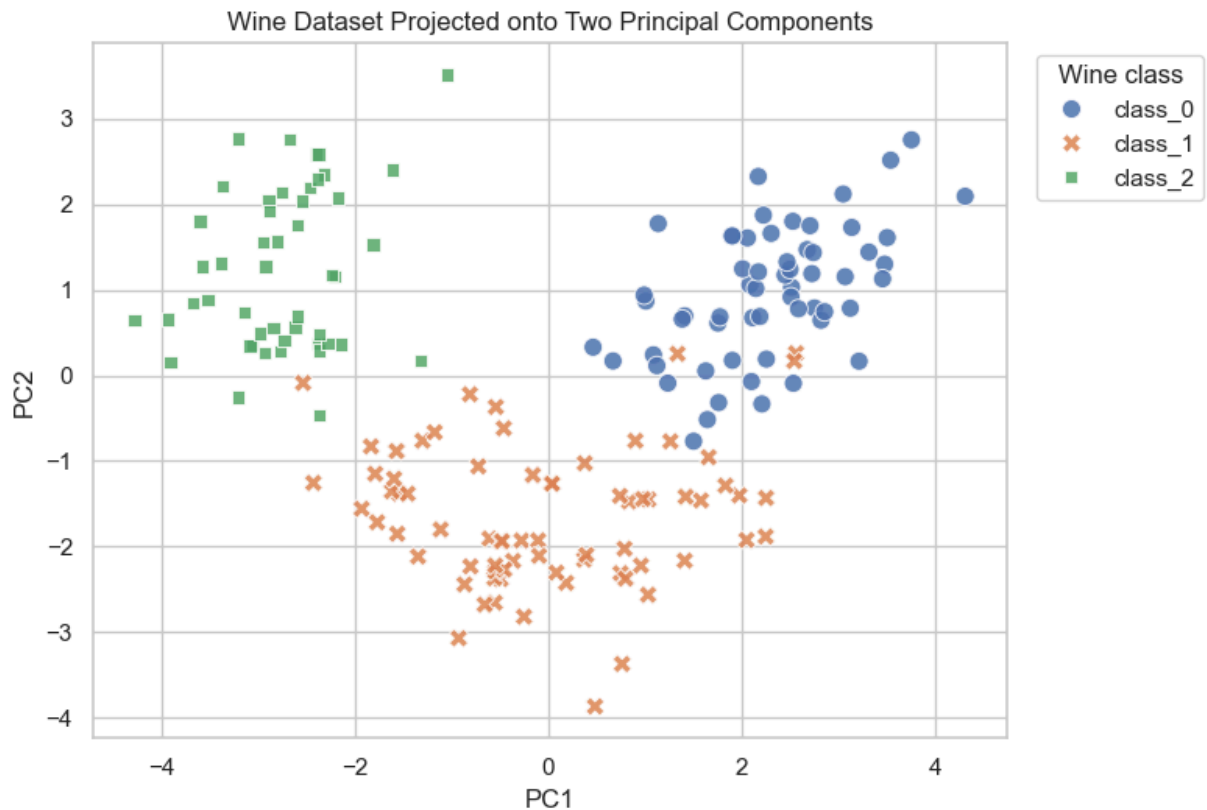
print("Explained variance ratio of PC1 and PC2:", np.round(pca_2.explained_v
print(f"Total variance retained in 2D: {pca_2.explained_variance_ratio_.sum(
```



```
plt.figure(figsize=(8, 5.5))
sns.scatterplot(data=pca_df, x="PC1", y="PC2", hue="Class", style="Class", s=50)
plt.title("Wine Dataset Projected onto Two Principal Components")
plt.legend(title="Wine class", bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()
```

Explained variance ratio of PC1 and PC2: [0.362 0.1921]

Total variance retained in 2D: 55.41%



```
In [9]: loadings = pd.DataFrame(
    pca_2.components_.T, index=X_wine.columns, columns=["PC1", "PC2"]
)
for component in loadings.columns:
    print(f"Largest absolute loadings for {component}:")
    display(loadings[component].reindex(loadings[component].abs().sort_values(
        ascending=False).index[:5]))

dataset_comparison = pd.DataFrame({
    "Original standardized data": [13, "100%", "Higher"],
    "PCA transformed data": [2, f"{pca_2.explained_variance_ratio_.sum():.2%}"],
    index=["Number of features", "Information retained", "Later computation c
display(dataset_comparison)
```

Largest absolute loadings for PC1:

PC1	
flavanoids	0.422934
total_phenols	0.394661
od280/od315_of_diluted_wines	0.376167
proanthocyanins	0.313429
nonflavanoid_phenols	-0.298533

Largest absolute loadings for PC2:

PC2	
color_intensity	0.529996
alcohol	0.483652
proline	0.364903
ash	0.316069
magnesium	0.299634

	Original standardized data	PCA transformed data
Number of features	13	2
Information retained	100%	55.41%
Later computation cost	Higher	Lower

PCA interpretation

- **PC1** captures the largest overall variation and is strongly influenced by phenol- and flavonoid-related measurements.
- **PC2** captures the next independent direction and is strongly influenced by alcohol and color-related measurements.
- Nearby points have similar standardized chemical profiles.
- Sign flips do not change a component's meaning; loading magnitude shows importance.

Advantages, limitations, and applications

Advantages

- Reduces feature count, noise, storage, and later computation.
- Removes multicollinearity because components are uncorrelated.
- Makes high-dimensional data easier to visualize.

Limitations

- Components are less interpretable than original features.
- PCA is linear and can miss nonlinear structure.
- It ignores class labels and is sensitive to feature scaling and outliers.

Applications

- Visualization, compression, noise reduction, preprocessing, and feature extraction.

Extra Credit: Linear Discriminant Analysis (LDA)

LDA uses the known class labels and searches for directions that separate the classes.

With three classes, it can produce at most two discriminants.

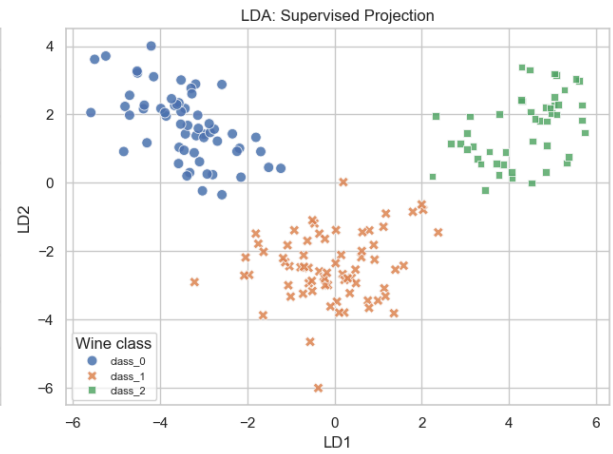
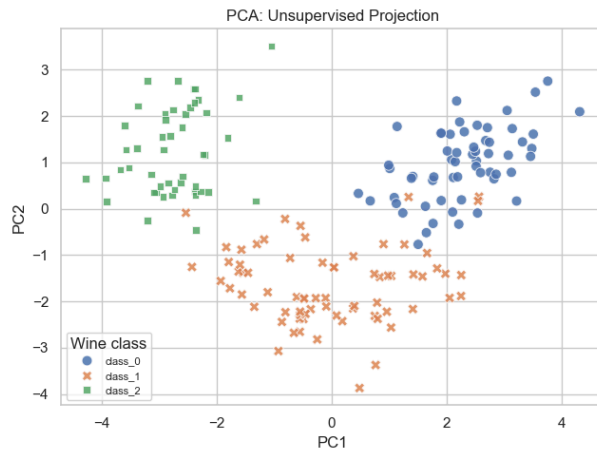
```
In [10]: lda = LinearDiscriminantAnalysis(n_components=2)
X_lda = lda.fit_transform(X_wine_scaled, y_wine)
lda_df = pd.DataFrame(X_lda, columns=["LD1", "LD2"])
lda_df["Class"] = y_wine.map(dict(enumerate(wine.target_names)))

print("LDA explained discriminative variance ratio:", np.round(lda.explained

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.scatterplot(data=pca_df, x="PC1", y="PC2", hue="Class", style="Class", s
axes[0].set_title("PCA: Unsupervised Projection")
sns.scatterplot(data=lda_df, x="LD1", y="LD2", hue="Class", style="Class", s
axes[1].set_title("LDA: Supervised Projection")
for ax in axes:
    ax.legend(title="Wine class", fontsize=8)
plt.tight_layout()
plt.show()

separation = pd.DataFrame({
    "Method": ["PCA", "LDA"],
    "Uses class labels": ["No", "Yes"],
    "2D silhouette score": [silhouette_score(X_pca, y_wine), silhouette_scor
})
display(separation.style.format({"2D silhouette score": "{:.4f}"}))
```

LDA explained discriminative variance ratio: [0.6875 0.3125]



	Method	Uses class labels	2D silhouette score
0	PCA	No	0.5262
1	LDA	Yes	0.6632

```
In [11]: print("Final observations:")
print(f"- PCA keeps {pca_2.explained_variance_ratio_.sum():.2%} of total var")
print(f"- {min_components_95} principal components are required to retain at")
print(f"- PCA preserves overall variation without using class labels.")
print(f"- LDA uses labels and gives clearer class separation in two dimension")
print(f"- PCA is suitable for general compression; LDA is useful when class s
```

Final observations:

- PCA keeps 55.41% of total variance in two components.
- 10 principal components are required to retain at least 95% variance.
- PCA preserves overall variation without using class labels.
- LDA uses labels and gives clearer class separation in two dimensions.
- PCA is suitable for general compression; LDA is useful when class separation is the goal.